# 네트워크 개요

# 1. 네트워크 기초

# 01 Client-Server Architecture

## 개념

- 복잡하고 비용이 큰 연산을 서버로 집중시키고, 클라이언트는 요청만 수행하도록 분리한 구조

## 배경

- Machine are expensive
  - 과거에는 고성능 컴퓨터가 매우 비쌌음
  - 모든 사용자가 고성능 장비를 갖는 것이 비효율적
- Application are complex
  - 어플리케이션의 기능이 점점 복잡해짐
  - 계산량 증가, 데이터 처리 증가, 유지보수 어려움

## 기본 아이디어

- Separate the application into two components.
  - 어플리케이션을 Client와 Server로 분리

| 구분 | 역할 |
|---|---|
| Client | 사용자 인터페이스, 요청 전송 |
| Server | 비즈니스 로직, 데이터 처리, 연산 |

# 01 Client-Server Architecture

## 역할 분담의 장점

- Expensive workload can be done on the server
  - 복잡한 계산, 대용량 데이터 처리
  - 중앙 집중식 관리 가능
  - 서버만 고성능 장비로 구성하면 됨
- Client call servers to perform expensive tasks
  - 클라이언트는 요청(Request)만 전달
  - 서버는 처리 후 결과(Response) 반환

## RPC 등장

- Remote Procedure Call (RPC)
  - 원격에 있는 서버의 함수를 로컬 함수처럼 호출하기 위해 등장

## RPC 특징

- 함수 중심 인터페이스
- 요청/응답 구조
- 직렬화(Serialization) 필요
- 네트워크 오류 가능성 존재

# 01 Client-Server Architecture

## 현대적 확장

- RPC 개념 이후로 발전된 기술
  - REST API
  - gRPC
  - MSA ( MicroServices Architecture)
  - Cloud / Distributed Systems

# 02 통신 모델

## 통신 모델 ( Communication Model )

- 통신 모델은 네트워크 통신을 계층(layer)별로 분리하여 표준화한 개념적 구조이다.
- **대표적으로 OSI 7계층 모델, TCP/IP 모델이 있으며 이 모델들은 복잡한 네트워크 통신을 이해, 설계, 개발, 운영하기 쉽게 만들어준다.**

## 주요 장점

- **어플리케이션의 네트워크 독립성 ( Agnostic Applications )**
  - 어플리케이션은 상위 계층(API) 만 사용
  - 실제 데이터 전송 방식은 하위 계층이 처리
  - 따라서 네트워크 매체가 바뀌어도 앱 코드는 변경 없음
  - ex. 집에서는 WIFI
  - 회사에서는 Ethernet
  - 이동 중에는 LTE/5G
  - 동일한 어플리케이션 코드로 모두 정상적으로 동작됨.

    만약 통신 모델이 없다면, 어플리케이션이 물리적인 네트워크 환경에 맞는 앱을 여러 번 개발해야 됨.

핵심정리:

통신 모델은 어플리케이션을 네트워크 기술로부터 분리하여 개발자가 네트워크 환경 변화에 신경 쓰지 않도록 해준다.

# 02 통신 모델

## 주요 장점

- **네트워크 장비 관리 및 확장성 ( Network Equipment Management )**
  - 각 장비는 특정 계층 역할만 수행
  - 장비 교체/업그레이드 시 다른 계층에 영향 최소화
  - 또한 서로 다른 제조사의 장비도 상호 운용 가능
  - ex. 스위치를 1Gbps 에서 10Gbps로 교체하면
    네트워크 성능만 향상되며 부수적인 어플리케이션 또는 서버 프로그램 수정이 필요 없음.
    만약 통신모델이 없다면 특정 어플리케이션, 특정 통신방식, 특정 제조사에 강하게 종속되기 때문에
    장비 업그레이드 시 전체 시스템을 수정하거나 서비스 중단 위험이 증가하게 되다.

핵심정리:
통신 모델은 네트워크 장비를 계층화하여 유지보수와 확장을 쉽고 안정하게 만든다.

# 02 통신 모델

## ■ 주요 장점

- **계층별 독립적 기술 발전 ( Decoupled Innovation )**
  - 각 계층은 독립적으로 발전 가능
  - 하위 계층에서의 변경 시 상위 계층 수정이 불필요
  - ex. 물리 계층: 구리선 > 광케이블 > 무선
    전송 계층: TCP 성능 개선
    응용 계층: HTTP/1.1 > HTTP/2 > HTTP/3
    서로 간에 영향을 거의 주지 않음
    만약 통신모델이 없다면 한 기술의 변경이 전체 시스템에 연쇄적으로 영향을 미치게 되어 새로운 기술 도입이 매우 어려워진다.

핵심정리:
통신 모델의 계층 구조는 각 기술이 독립적으로 혁신될 수 있는 기반을 제공한다.

# 02 통신 모델

## ■ 주요 장점

### ▪ 문제 분석 및 트러블슈팅 용이 ( Ease of Troubleshooting and Problem)

- 각 계층이 명확한 역할을 가짐
- 문제가 발생한 계층 범위를 빠르게 한정 가능하기 때문에 장애 복구 시간 단축 가능
- ex. 케이블 단선이면 물리 계층 문제.
  IP 통신 불가이면 네트워크 계층 문제.
  웹 화면이 안 보이면 응용 계층 문제.
  만약 통신모델이 없다면 장애가 발생했을 때, 문제 원인을 찾기가 쉽지 않게 됨.

핵심정리:
네트워크 기능을 계층(layer)로 분리함으로써 문제 발생 시 체계적이고 효율적인 분석과 트러블슈팅을 가능하게 한다.

# 02 OSI 모델

## OSI 모델 ( Open Systems Interconnection Model )

- 네트워크 통신 과정을 7개의 계층(Layer)으로 나누어 표준화한 개념이다.
- 각 계층은 특정한 네트워크 기능을 담당하며, 계층별로 역할을 분리함으로써 이해, 설계, 문제 해결을 쉽게 처리.

## OSI 모델이 필요한 이유

- 네트워크 통신은 단순히 데이터를 보내고 받는 수준이 아니고 다음과 같은 복잡한 문제들을 포함한다.
- 이러한 복잡성을 계층(Layer)으로 나누어 표준화 및 역할 분리를 한 것이 OSI 모델이다.
  - 데이터는 어떤 형식으로 표현할 것인가?
  - 통신을 시작하고 종료하는 기준은 무엇인가?
  - 데이터가 손실되면 재전송해야 하는가?
  - 목적지까지 어떤 경로로 보낼 것인가?
  - 실제 전선이나 무선으로 어떻게 전달되는가?

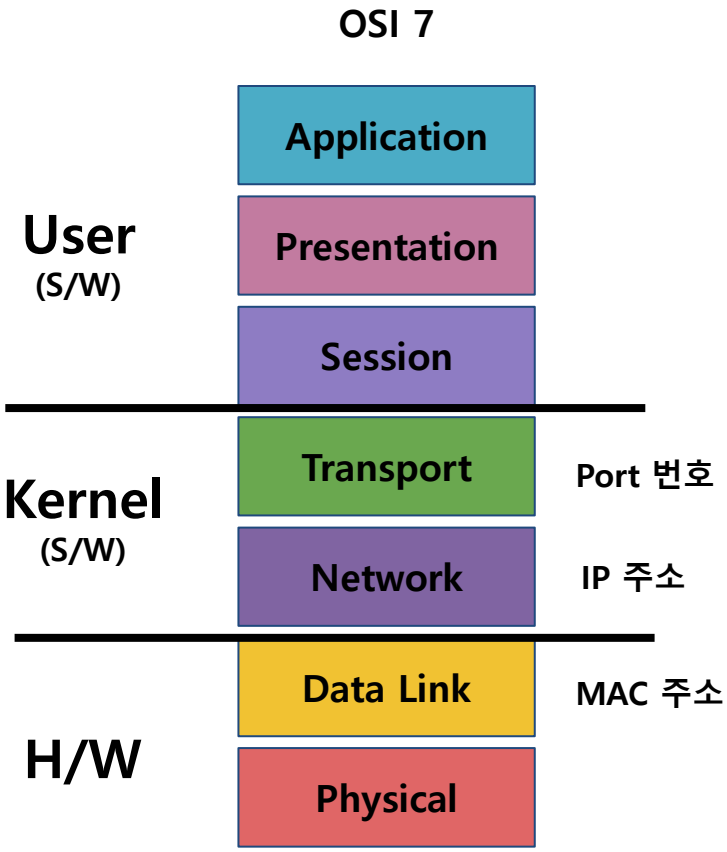
## OSI 모델 핵심

- **각 계층은 자신의 역할만 책임.**
- **상위 계층은 하위 계층의 내부 구현을 모름.**
- **문제 발생 시 어느 계층 문제인지 빠르게 판단 가능.**

# 03 OSI 모델 7계층

IP 번호

- **Layer 7 – Application Layer (응용 계층)**
  - **HTTP**/FTP/gRPC
- **Layer 6 – Presentation Layer (표현 계층)**
  - 직렬화(Serialization), 인코딩(Encoding)
- **Layer 5 – Session Layer (세션 계층)**
  - 세션 연결, TLS
- **Layer 4 – Transport Layer (전송 계층)**
  - UDP/**TCP**
- **Layer 3 – Network Layer (네트워크 계층)**
  - **IP**
- **Layer 2 – Data Link Layer (데이터 링크 계층)**
  - Frames, Mac address
- **Layer 1 – Physical Layer (물리 계층)**
  - 전기 신호 (0|1)



# 03 OSI 모델 7계층

## Layer 7 – Application Layer (응용 계층)

- 역할
  - 사용자 어플리케이션이 **네트워크 기능을 사용하는 인터페이스**
  - 실제 데이터를 주고받는 **프로토콜이 정의됨**
- **주요 기능**
  - **요청(Request) / 응답(Response)**
  - **파일 전송, 이메일, 웹 통신 등**
- 대표 프로토콜
  - **HTTP /HTTPS**
  - FTP / SFTP
  - SMTP / POP3 / IMAP
  - DNS
- 대표 예시
  - 웹 브라우저에서 웹 주소 입력 ( https://example.com )

# 03 OSI 모델 7계층

## Layer 6 – Presentation Layer (표현 계층)

- 역할
  - 데이터의 **형식과 표현** 방식 담당
  - 서로 다른 시스템 간 데이터 호환성 보장
- **주요 기능**
  - **인코딩 / 디코딩**
  - **암호화 / 복호화**
  - **압축 / 해제**
  - **직렬화 / 역직렬화**
- 대표 예시
  - 문자 인코딩: UTF-8, ASCII
  - 데이터 포맷: JSON, XML

# 03 OSI 모델 7계층

- **Layer 5 – Session Layer (세션 계층)**
  - 역할
    - 통신 세션 관리
    - 연결의 시작, 유지, 종료 담당
  - 주요 기능
    - 세션 생성 및 종료
    - 동기화
  - 대표 예시
    - 로그인 세션
    - TLS 핸드셰이크
    - 장시간 유지되는 연결 관리

# 03 OSI 모델 7계층

## Layer 4 – Transport Layer (전송 계층)

- 역할
  - 종단 간(End-to-End) 데이터 전송
  - 신뢰성, 순서 보장, 오류 제어
- **주요 기능**
  - **포트 번호 사용**
  - 흐름 제어 ( Flow Control)
  - 오류 제어
  - 재전송
- 대표 프로토콜
  - **TCP : 신뢰성, 순서 보장**
  - **UDP: 빠르지만 비신뢰적**

- 대표 예시
  - 웹 서비스: TCP
  - 스트리밍/게임: UDP

# 03 OSI 모델 7계층

## Layer 3 – Network Layer (네트워크 계층)

- 역할
  - 논리적 주소(IP) 기반 전달
  - 서로 다른 네트워크 간 통신
- **주요 기능**
  - **IP 주소 지정**
  - **라우팅**
  - **패킷 전달**
- 대표 개념
  - **IPv4 / IPv6**
  - 라우터
  - 경로 선택 알고리즘
- 대표 예시
  - ping
  - 서로 다른 대역 간 통신

# 03 OSI 모델 7계층

## Layer 2 – Data Link Layer (데이터 링크 계층)

- 역할
  - 같은 네트워크(Local Network) 내 전달
  - 프레임 단위 통신
- **주요 기능**
  - **MAC 주소 사용**
  - 오류 검출
  - 프레임 생성
- 대표 개념
  - **Ethernet**
  - MAC Address
  - ARP
  - 스위치
- 대표 예시
  - 같은 사무실 PC 간 통신
  - 스위치에서 MAC 주소 기반 전달

# 03 OSI 모델 7계층

- **Layer 1 – Physical Layer (물리 계층)**
  - 역할
    - 실제 신호 전송
    - 비트(0, 1)를 물리적으로 전달
  - 주요 요소
    - 전기 신호
    - 광 신호
    - 무선 신호
  - 대표 예시
    - UTP 케이블
    - 광섬유
    - Wi-Fi

# 04 OSI 모델 – 송신자 (Sender) 관점

## 전체 흐름

- 웹 페이지에서 HTTPS로 POST 요청을 보내는 경우이다.
- "어플리케이션에서 생성된 POST 요청은 OSI 7계층에서 시작해서 1계층까지 내려가며 점점 캡슐화되어 실제 신호로 전송된다."

## Layer 7 – Application Layer ( 응용 계층 )

- 동작
  - 어플리케이션이 **HTTPS 로 POST 요청 생성**
  - 요청 본문(body)에 **JSON 데이터 포함**
- 핵심 포인트
  - 아직 네트워크 전송 방식(TCP/IP)은 모름
  - 단지 "무엇을 보내는지 " 만 정의
- 예시
  - 브라우저, axios, fetch, curl 이용
  - POST /api/orders { "item": "book", "qty": 1 }

# 04 OSI 모델 – 송신자 (Sender) 관점

## Layer 6 – Presentation Layer ( 표현 계층 )

- 동작
  - JSON 객체를 **바이트 문자열(byte stream)**로 변환
  - 직렬화 (Serialization)
  - 문자 인코딩 적용 (UTF-8)
- 핵심 포인트
  - 서로 다른 시스템이 같은 의미로 데이터를 해석하도록 형식 통일
  - 암호화도 개념적으로 이 계층에 포함됨
- 예시
  - {"id": 1} 형식을 7b 22 69 64 22 3a 31 7d 로 변환

## Layer 5 – Session Layer ( 세션 계층 )

- 동작
  - **연결을 만들기 위한 요청**
  - TCP 연결 + TLS 세션 수립 요청
- 핵심 포인트
  - "요청 서버와 대화를 시작하겠다"는 논리적 개념
  - 항상 명확히 분리되어 구현되지는 않음
- 예시
  - HTTPS 통신을 위한 TLS 세션 준비
  - 기존 세션 재사용 가능 여부 판단

# 04 OSI 모델 – 송신자 (Sender) 관점

- **Layer 4 – Transport Layer ( 전송 계층 )**
  - 동작
    - TCP 연결 시작
    - 대상 포트 443(HTTPS)로 SYN 패킷 전송
  - 핵심 포인트
    - 포트 번호 사용
    - 아직 데이터 전송 전 단계
    - 신뢰성 확보를 위한 연결 수립
  - 예시
    - SYN -> SYN-ACK -> ACK
    - 이 단계가 끝나야 실제 POST 데이터 전송 가능
- **Layer 3 – Network Layer ( 네트워크 계층 )**
  - 동작
    - TCP SYN 세그먼트를 **IP패킷에** 담음
    - 출발지 IP / 목적지 IP 추가
  - 핵심 포인트
    - "어디로 보낼 것인가" 결정
    - 라우터는 이 IP 정보를 보고 전달
  - 예시
    - 소스 IP : 192.168.0.10
    - 타겟 IP: 142.250.XXX,XXX

# 04 OSI 모델 – 송신자 (Sender) 관점

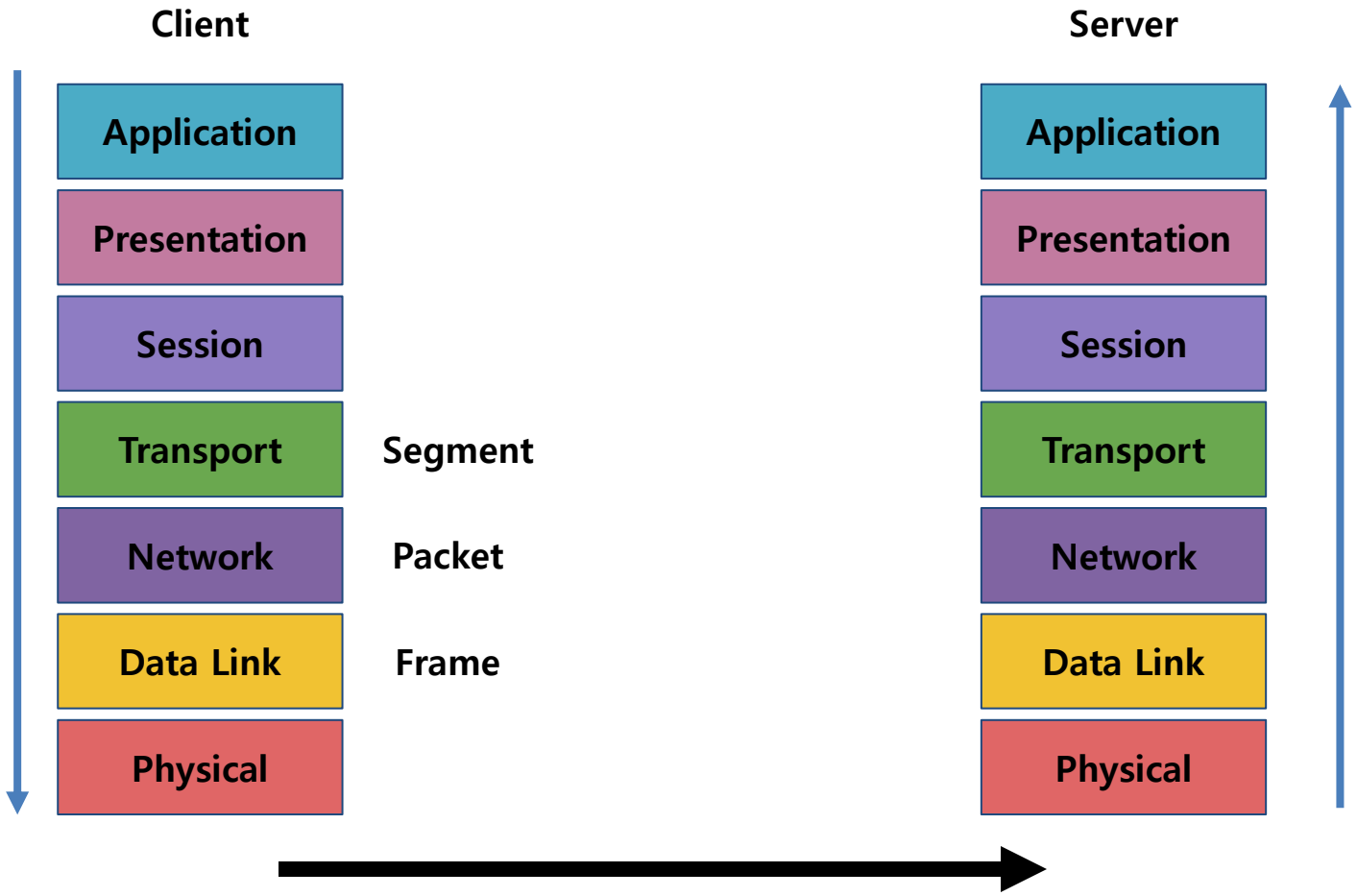
## Layer 2 – Data Link Layer ( 데이터 링크 계층 )

- 동작
  - IP 패킷을 **프레임(Frame)**으로 감쌈
  - 출발지 MAC / 목적지 MAC 주소 추가
- 핵심 포인트
  - 같은 네트워크 구간에서의 실제 전달
  - 목적지 MAC이 같은 LAN이면 상대 장비 MAC, 다른 네트워크면 라우터(게이트웨이) MAC
- 예시
  - 소스 MAC: 내 NIC
  - 타겟 MAC: 공유기 MAC

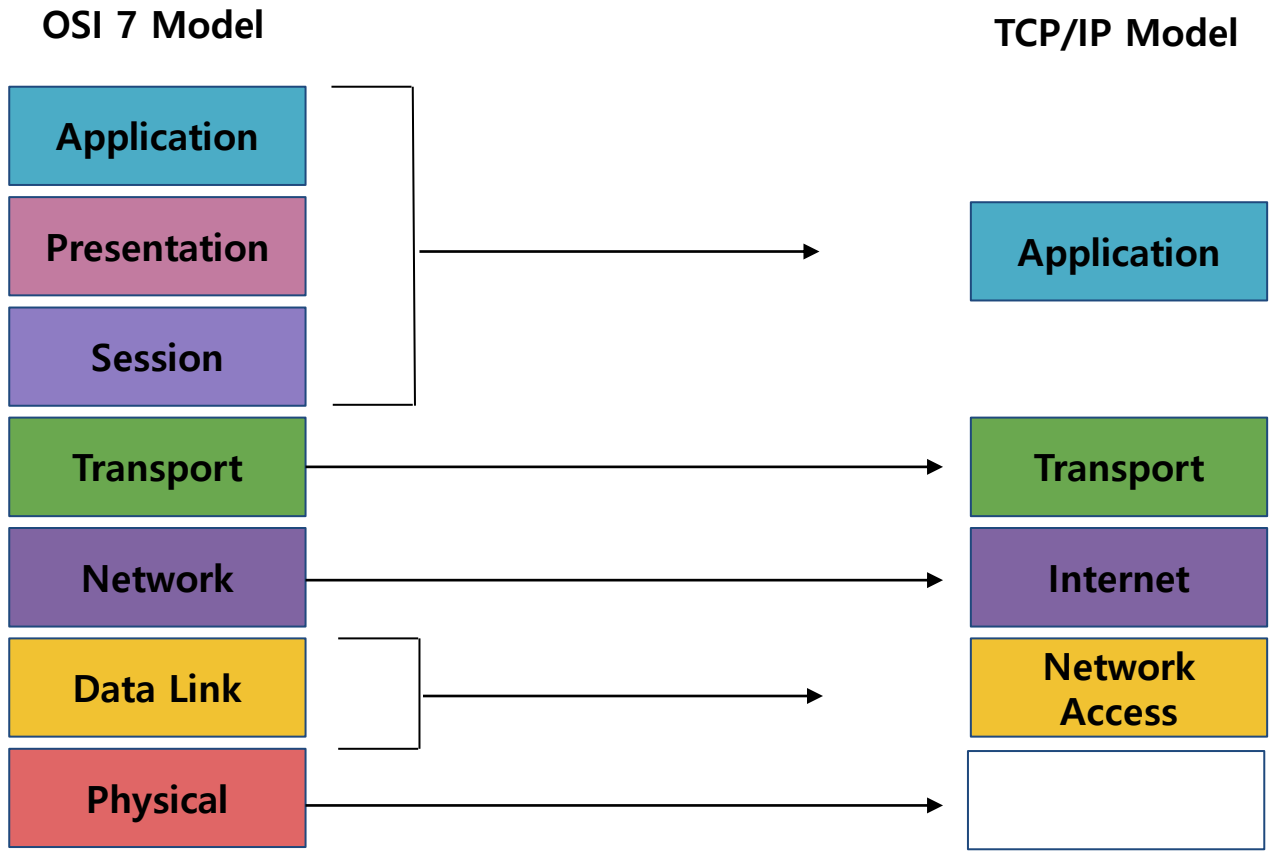
## Layer1– Physical Layer ( 물리 계층 )

- 동작
  - 프레임이 비트열 (0과 1)로 변환
  - 물리 매체를 통해 신호로 전송
- 핵심 포인트
  - 이 단계부터는 완전히 물리적 현상
- 예시
  - Wi-Fi : 전파 (Radio signal)
  - Ethernet : 전기 신호
  - Fiber: 빛(Light)

# 05 HTTPS POST 요청/응답



# 06 TCP/IP 모델 ( 4계층 )



# 2. 인터넷 프로토콜 (IP)

# 01 기본 장치

## 호스트(host)

- 트래픽을 송수신하는 모든 장치 ( 인터넷 연결됨 )
- ex. 컴퓨터, 핸드폰, 프린터기, 서버, IoT 기반의 모든 장치들(TV, 냉장고 .. )

## 허브(hub)

- 동일 네트워크내에서 여러 호스트들을 연결 할 수 있는 장치.
- 연결된 모든 호스트들은 다른 호스트들의 데이터를 broadcast 방식으로 수신됨.

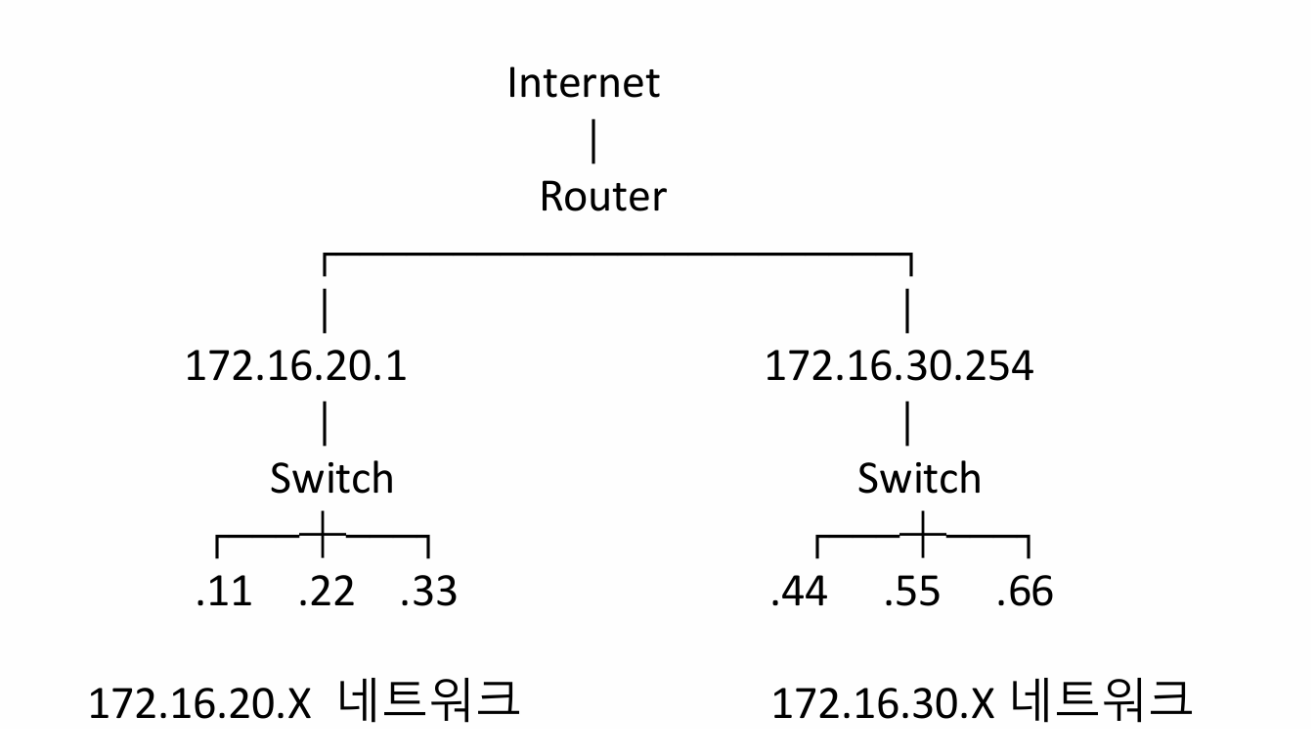
## 스위치(switch)

- 허브 + unicast 기능
- 원하는 특정 호스트에만 데이터 전달이 가능해짐

## 라우터(router)

- 서로 다른 네트워크간 통신을 처리하는 장치
- 네트워크 경계에 있기 때문에 보안 정책을 적용하기에 알맞은 논리적인 지점
- 호스트가 라우터에 접근해야 되기 때문에 라우터 자신도 IP 주소를 가지며, 이 IP주소는 게이트웨이(Gateway)역할을 하게 됨
- 즉, 게이트웨이(Gateway)는 호스트가 로컬 네트워크를 나가는 통로임
- 라우터가 라우팅 할 수 있도록 경로(path) 정보가 필요 ( 라우트 테이블(route table) 이용 )

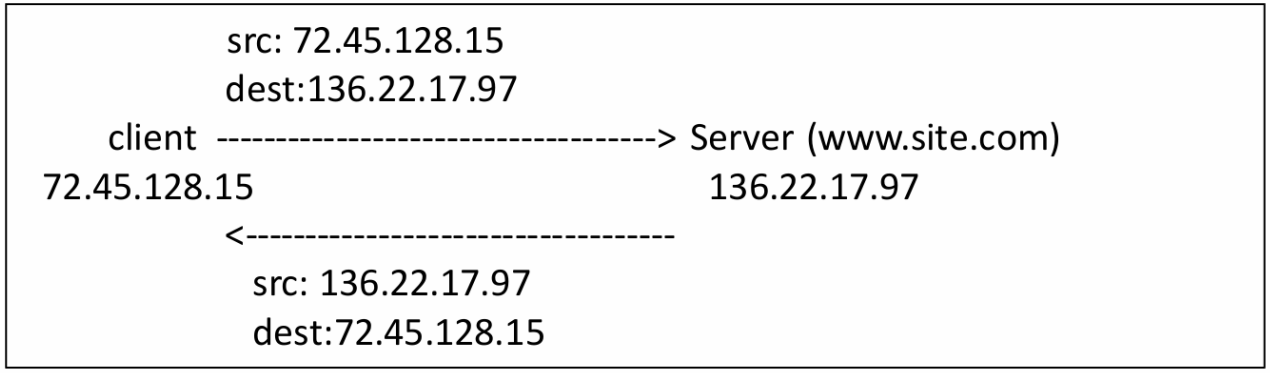
# 01 기본 장치


Internet
Router
172.16.20.1
172.16.30.254
Switch
Switch
.11
.22
.33
.44
.55
.66
172.16.20.X 네트워크
172.16.30.X 네트워크

# 02 IP Address

## 개요

- 인터넷에 연결된 장치인 Host를 식별하기 위한 논리적 주소
- OSI 7 계층 중 Layer 3(Network Layer)에 속함
- 라우터(Router)가 이 주소를 보고 패킷을 전달함
- 자동 설정 및 수동 설정 모두 가능
- IPv4 기준으로 32 bit로 구성 ( 4개의 octet으로 구분 ) ex. 10.10.0.0
- 각 호스트가 보내는 데이터에  src와 dest가 포함됨.



# 02 IP Address

- IP 주소는 두 부분으로 나뉜다.
  - Network + Host
  - 예시> **192.168.254.10 /24**

    Network 부분: **192.168.254**

    Host 부분: **10**

| IP | 결과 |
| --- | --- |
| **192.168.254.**10 | 같은 네트워크 |
| **192.168.254.**200 | 같은 네트워크 |
| **192.168.253.**10 | 다른 네트워크 ( 라우터 필요) |

# 02 Subnet Mask

## 개요

- IP 주소에서 **Network 부분과 Host 부분을 구분하기 위한 기준값**
- IP 주소 혼자서는 "어디까지가 네트워크고 어디부터가 호스트인지" 알 수 없음
  따라서 그 기준을 알려주는 것이 Subnet Mask 임
- **Subnet Mask 활용하여 특정 IP들이 같은 네트워크에 있는지 알 수 있음 (중요)**
  - **같은 Subnet : MAC 주소로 직접 통신 가능**
  - **다른 Subnet:  라우터(게이트웨이) 이용해서 통신 가능**

## CIDR 표기법

- 표기법 예: 192.168.254.0 **/24**
- **IPv4는 총 32비트로 표기** ex. **0~255(8비트). 0~255(8비트). 0~255(8비트).** 0~255(8비트)
- /24는 앞 24비트가 Network 부분임을 의미하고 8비트 모두 1로 설정.
  그래서 **255.255.255.0** 으로 나옴. 이 값을 Subnet Mask 라고 부름.

# 02 Subnet Mask

## 같은 네트워크 식별 방법

- **Network 주소 = IP 주소와 Subnet Mask를 AND 연산**
- 실제 계산:

  IP 주소: 192.168.254.10 , 192.168.254.200, 192.168.253.10

  **Subnet Mask: 255.255.255.0**

  제공된 IP주소와 Subnet Mask를 모두 2진수로 변환

  11000000 . 10101000 . 11111110 . 00001010 (192.168.254.10 )

  11000000 . 10101000 . 11111110 . 11001000 (192.168.254.200 )

  11000000 . 10101000 . 11111101 . 00001010 (192.168.253.10 )

  11111111 . 11111111 . 11111111 . 00000000 (255.255.255.0)

---

AND 연산 결과

11000000 .10101000 .111111**10** .00000000

11000000 .10101000 .111111**01** .00000000

10 진수로 변환

192.168.**254**.0

192.168.**253**.0

의미: 192.168.254.10 와 192.168.254.200 모두 동일한 Network 주소를 가짐.

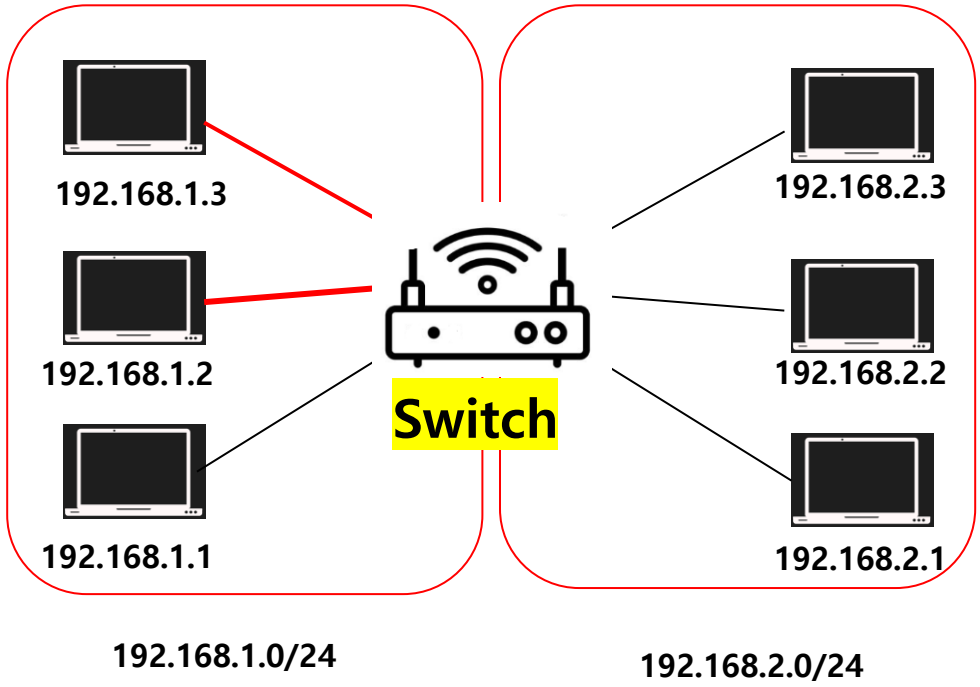
**즉 같은 Subnet(192.168.254.0/24) 에 속한다고 할 수 있다.**

**하지만 192.168.253.10 은 다른 Subnet(192.168.253.0/24)에 속한다고 할 수 있다.**

# 02 Subnet Mask

## 같은 네트워크 통신

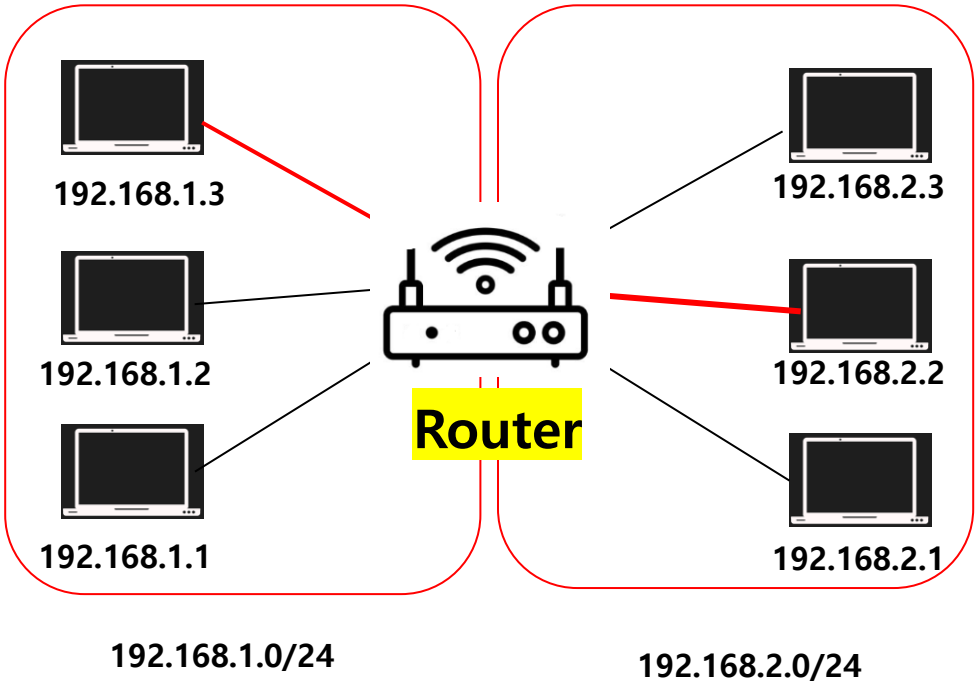
- 192.168.1.3 에서 192.168.1.2 로 요청 가정.
- 앞서 실습했던 Subnet Mask를 이용하여 동일한 Network 인지를 판별한다.
- **192.168.1.3** & 255.255.255.0 = **192.168.1.0**
- **192.168.1.2** & 255.255.255.0 = **192.168.1.0**
- **실행결과 동일한 Subnet 이기 때문에 Router 없이 동작 가능.**



# 02 Subnet Mask

## 다른 네트워크 통신

- 192.168.1.3 에서 192.168.2.2 로 요청 가정.
- 앞서 실습했던 Subnet Mask를 이용하여 동일한 Network 인지를 판별한다.
- **192.168.1.3** & 255.255.255.0 = **192.168.1.0**
- **192.168.2.2** & 255.255.255.0 = **192.168.2.0**
- **실행결과 동일한 Subnet 이 아니기 때문에 Router 필요함.**



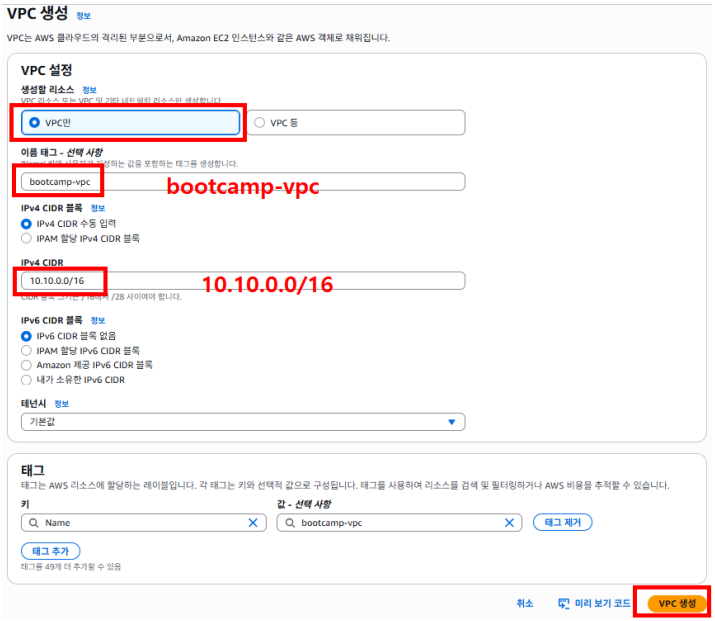
# 03 Private Network ( 사설망 )

## 개념

- 한정된 공인 IP 주소를 최대한 활용하기 위해 사설 IP주소로 만든 망.
- IPv4 기준으로 최대 IP 주소 개수는 43억개까지만 지원 가능 (4,294,967,296 개)
- RFC 1918 문서 ( 사설 IP 주소의 표준을 정의 )

## 특징

- 사설망 내부에서는 외부 인터넷 망으로 통신이 불가능한 구조.
- 외부와 통신할 때는 통신 가능한 공인 IP로 변경해서 통신함 ( NAT: Network Address Translation 기술 이용)
- AWS에서 사용하는 VPC 도 사설망으로 되어 있음. ( VPC: Virtual Private Network )



# 04 NAT ( Network Address Translation )

## 개념

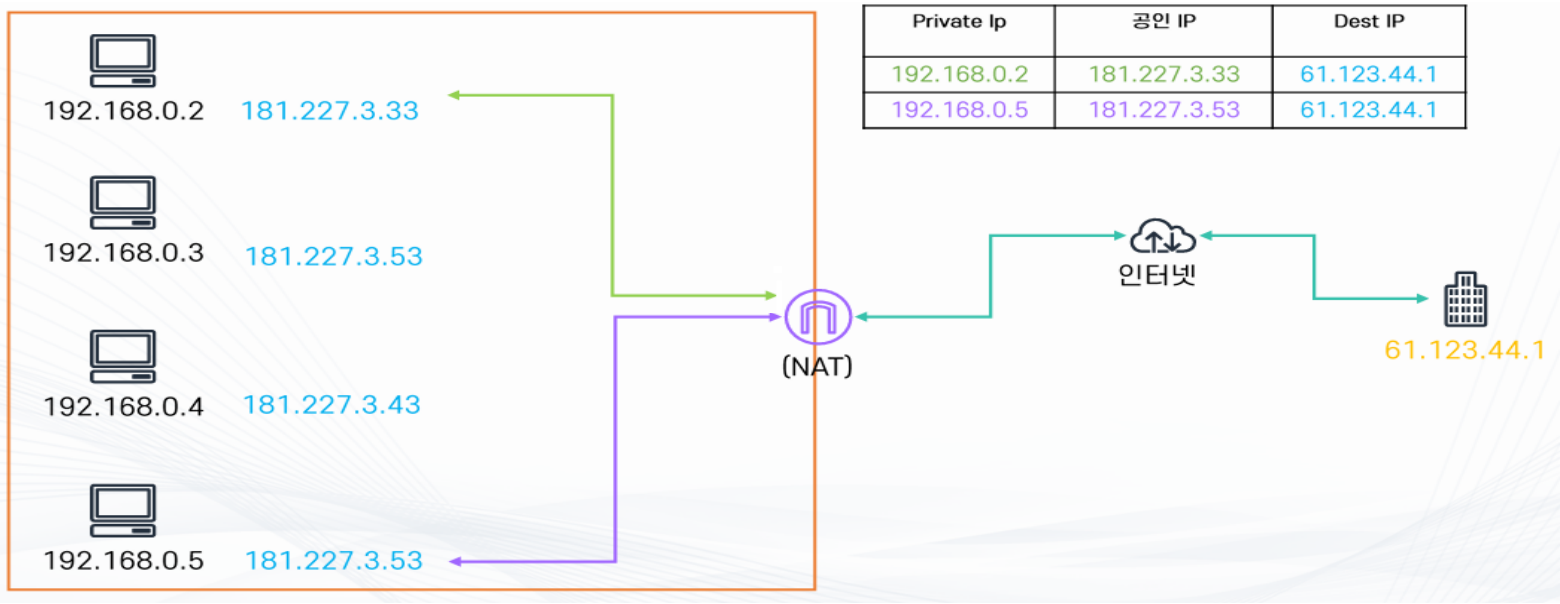
- 사설 IP ( Private IP )를 공인 IP (Public IP )로 변환하여 인터넷과 통신할 수 있도록 지원하는 기술.
- IPv4 주소 부족으로 내부에서는 Private IP를 사용하고 인터넷에서는 Public IP를 사용하기 위한 목적.
- 외부에서는 Private IP에 직접 접근이 불가능하기 때문에 보안효과 기대.

## 종류

- Static NAT : 하나의 사설 IP를 고정된 하나의 공인 IP로 연결, AWS Internet Gateway에서 사용하는 방식.
- Dynamic NAT: 하나의 사설 IP를 가용 가능한 공인 IP로 연결 (공인 IP 풀 이용 )
- PAT ( Port Address Translation ) : 많은 사설 IP를 하나의 공인 IP로 연결, NAT Gateway에서 사용하는 방식.

# 04 NAT ( Network Address Translation )

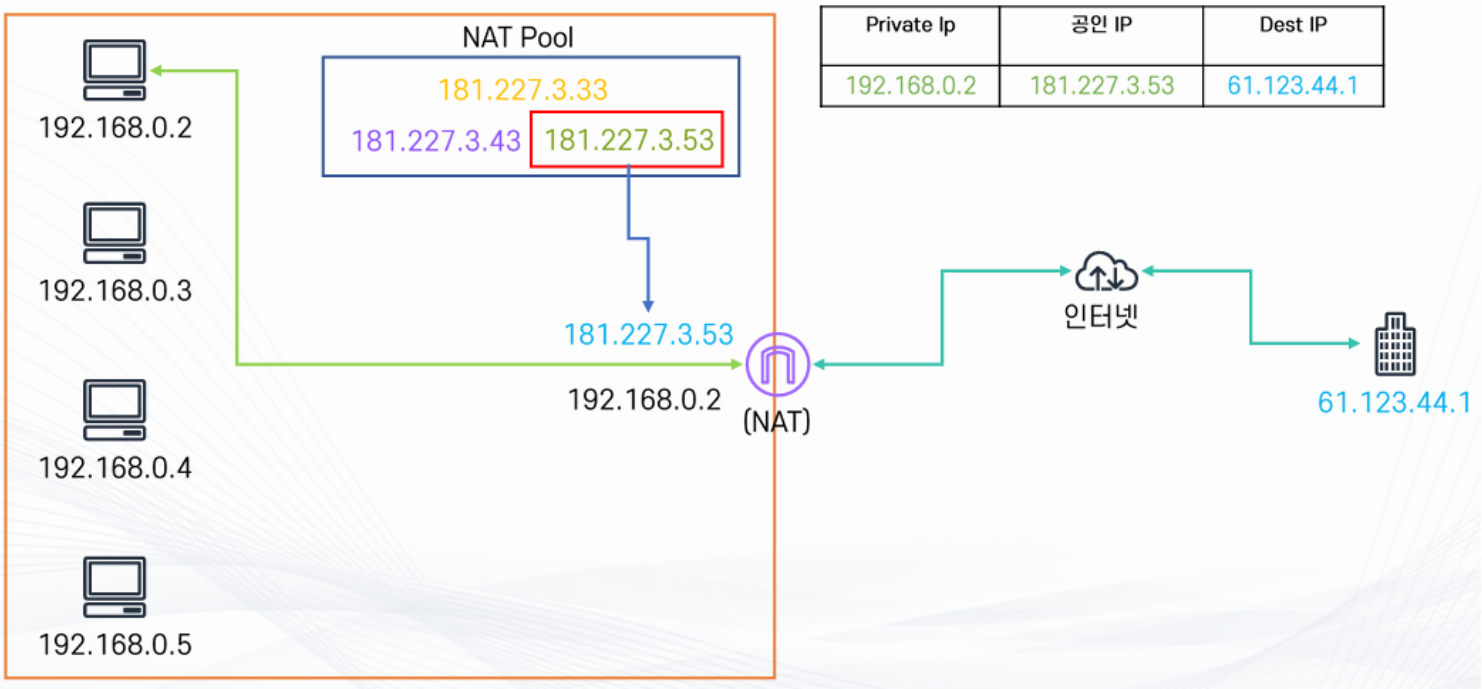
- **Static NAT**




| Private Ip | 공인 IP | Dest IP |
|---|---|---|
| 192.168.0.2 | 181.227.3.33 | 61.123.44.1 |
| 192.168.0.5 | 181.227.3.53 | 61.123.44.1 |

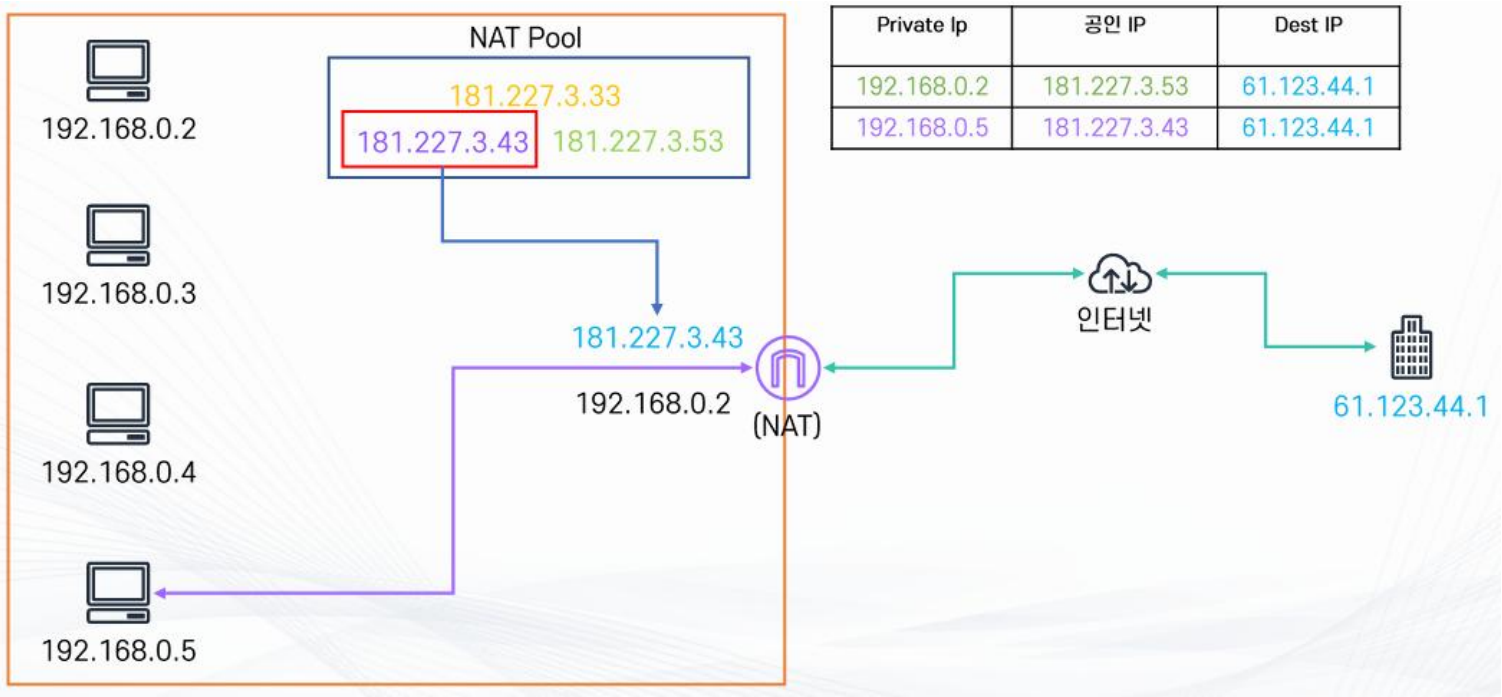
# 04 NAT ( Network Address Translation )

- **Dynamic NAT**



# 04 NAT ( Network Address Translation )

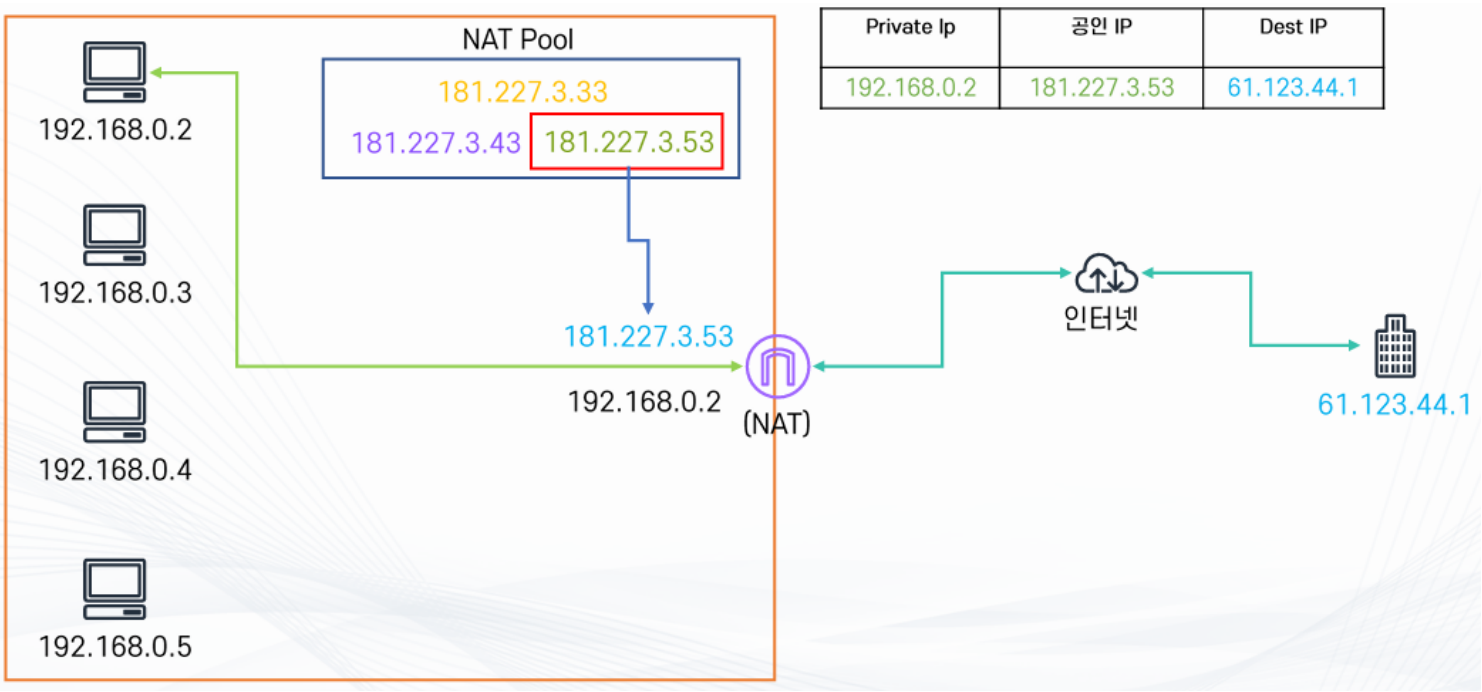
- **Dynamic NAT**




| Private Ip | 공인 IP | Dest IP |
|---|---|---|
| 192.168.0.2 | 181.227.3.53 | 61.123.44.1 |
| 192.168.0.5 | 181.227.3.43 | 61.123.44.1 |

# 04 NAT ( Network Address Translation )

- **PAT ( Port Address Translation )**



# 05 사설 IP 표기법

## 개념

- 1996년에 발표된 RFC 1918 문서에서 사설 IP 주소의 표준을 정의.

## 표기법 1 ( Classful IP 방식, 과거 )

- Class A
  - 사설 IP 범위: 10.0.0.0 ~ 10.255.255.255
  - IP 개수: 2^(32-8) = 16,777,215
  - 활용: 기업 네트워크, AWS

- Class B
  - 사설 IP 범위: 172.16.0.0 ~ 172.31.255.255 (172.0.0.0~172.15.255.255 범위는 Public IP임)
  - IP 개수: 2^(32-12) = 1,048,576
  - 활용: 관공서, 학교 등 네트워크

- Class C
  - 사설 IP 범위: 192.168.0.0 ~ 192.168.255.255
  - IP 개수: 2^(32-16) = 35,536
  - 활용: 가정용 공유기
  -

# 05 사설 IP 표기법

## 표기법 2 ( CIDR 방식, 현재 )

- CIDR ( Classless Inter Domain Routing )
  - 여러 개의 사설망을 구축하기 위해서 망을 나누는 방법으로, 네트워크와 호스트의 경계를 알려주는 표기법.
  - 낭비없이 각 호스트 주소 숫자만큼의 IP를 가진 네트워크 망 구축 가능.
  - 기존 Classful IP 방식은 매우 비효율적임. ( ex. 500개만 필요해도 Class C는 65,536 개가 만들어짐 )

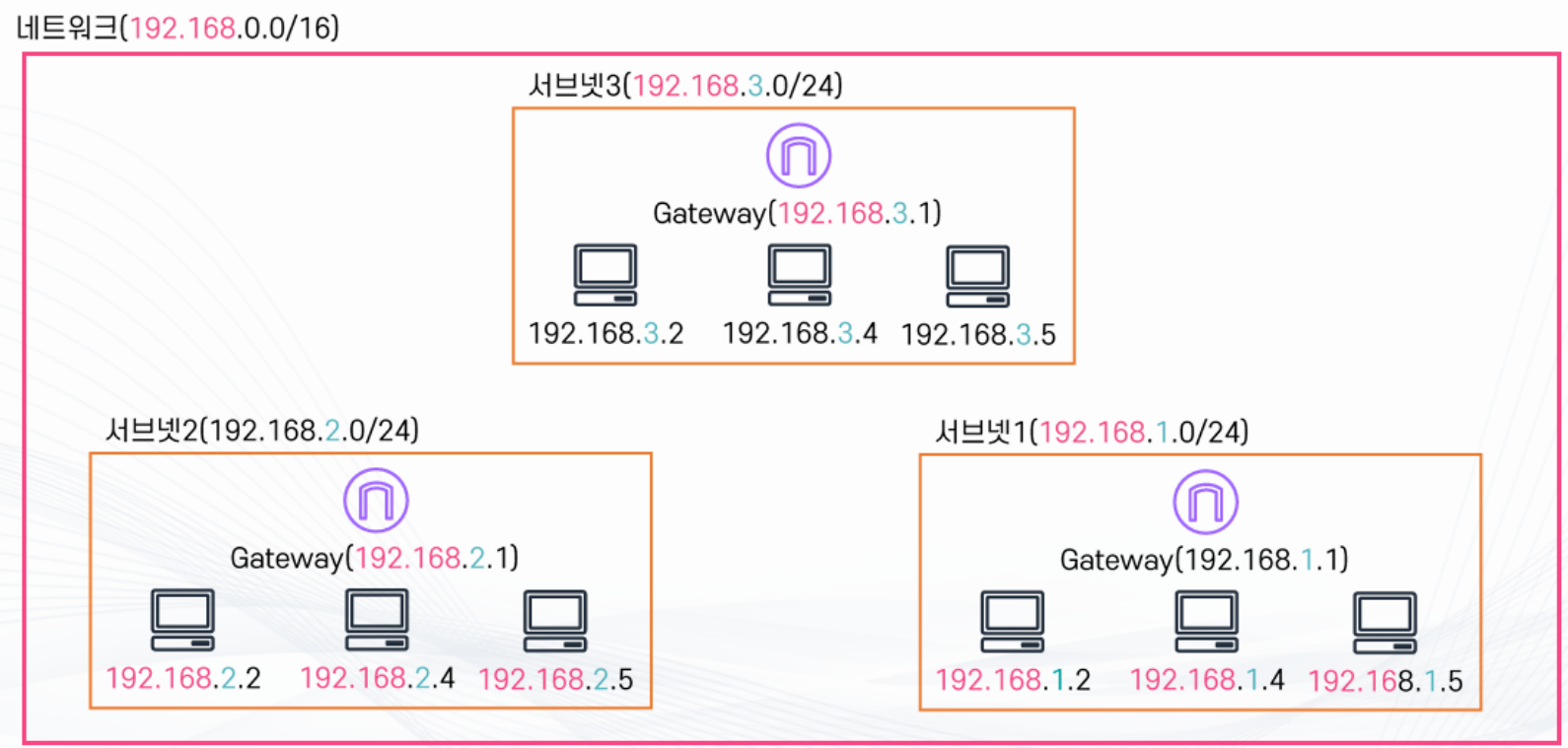
- 표기법
  - 문법: A.B.C.D/E 형식
  - A.B.C.D 에 IP 설정 ( octet )
  - E 에 네트워크 주소가 몇 bit인지 표시. 나머지가 호스트 주소임
  - ex. 10.0.0.0/16 , 172.16.0.0/12



# 06 서브넷 (Subnet)

## 개념

- 네트워크안의 네트워크로서, 큰 네트워크를 필요에 의해서 잘게 쪼갠 단위임.
- 서브넷은 일정 IP 주소의 범위를 보유함.
- 실제 사용 가능한 주소는 네트워크 주소와 브로드캐스트 주소 2개를 제외하고 사용가능.



# 05 사설 IP 표기법

- **CIDR 표기법 해석 가이드**

**1. 10진수를 2진수로 바꾼다.**

**2. CIDR 슬래시 뒤에 있는 숫자를 활용해서 2진수로 자른다.**

**3. 잘린 2진수의 왼쪽값은 그대로 고정시키고, 잘린 2진수의 오른쪽 값에서 표현할 수 있는 최소값과 최대값을 구한다.**

**4. 2진수로 표현된 최소값과 최대값을 10진수로 변환한다.**

**ex. 13.25.82.0/24**

**1. 10진수를 2진수로 바꾼다.**
**00001101.00011001.01010010.00000000**

**2. CIDR 슬래시 뒤에 있는 숫자를 활용해서 2진수로 자른다.**
**00001101.00011001.01010010 / 00000000**

**3. 잘린 2진수의 왼쪽값은 그대로 고정시키고, 잘린 2진수의 오른쪽 값에서 표현할 수 있는 최소/최대값을 구한다.**
**최소값: 00001101.00011001.01010010 / 00000000**
**최대값: 00001101.00011001.01010010 / 11111111**

**4. 2진수로 표현된 최소값과 최대값을 10진수로 변환한다.**
**최소값 : 13.25.82.0 ( 네트워크 주소 )**
**최대값: 13.25.82.255 ( 브로드캐스트 주소 )**
**사용 가능한 주소: 13.25.82.1 ~ 13.25.82.254 ( 254개 사용 가능 )**

# 05 사설 IP 표기법

**실습1: 25.212.157.0/25**

**네트워크 주소:**

**브로드캐스트 주소:**

**사용 가능한 주소 범위 및 개수 :**

**실습2: 25.212.157.128/25**

**네트워크 주소:**

**브로드캐스트 주소:**

**사용 가능한 주소 범위 및 개수 :**

**실습3: 37.5.13.192/26**

**네트워크 주소:**

**브로드캐스트 주소:**

**사용 가능한 주소 범위 및 개수 :**

# 3. UDP/TCP

# 01 UDP ( User Datagram Protocol )

## 개념

- **신뢰성 보다는 빠르게 보내는데 집중한 전송 계층 ( Layer 4 ) 프로토콜**.
- 사전 연결 설정 없이 데이터 보냄

## Layer 4 계층

- Layer 3 계층인 IP 역할은 어느 Host로 갈지를 결정.
- Layer 4 계층인 UDP/TCP는 그 Host 안에서 어떤 프로그램(S/W)으로 갈지를 결정 ( **Port 번호 활용** )

## 주요 용도

- 게임/스트리밍
- Zoom
- 실시간 음성/영상 통신 ( WebRTC )

# 01 UDP ( User Datagram Protocol )

## Multiplexing & Demultiplexing

- Multiplexing ( 송신 측)
  - 여러 앱의 데이터를 하나의 전송 계층 프로토콜인 UDP로 모아서 보낸다.
  - UDP가 여러 앱의 출발지 port / 목적지 port 를 저장한다.

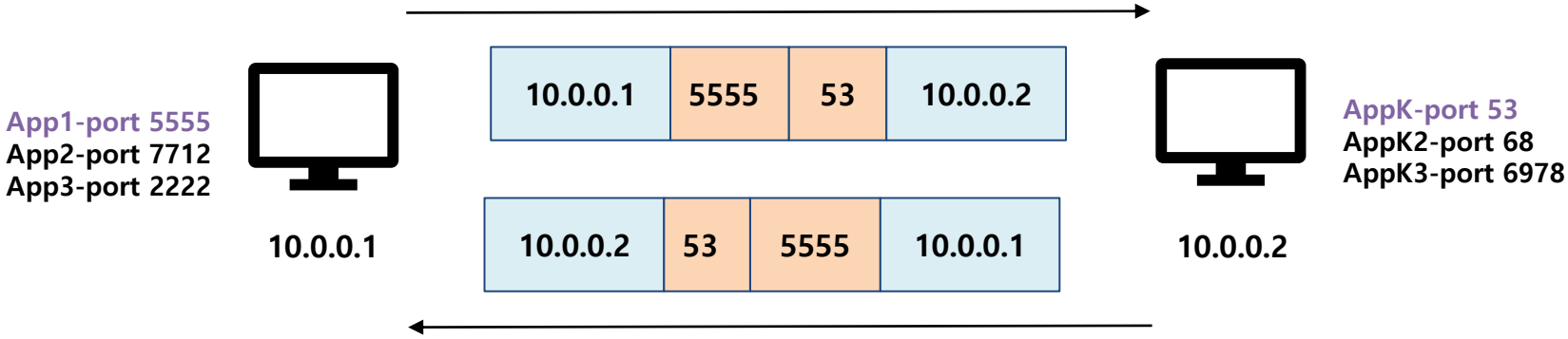
```
[App1] ┐
[App2] ├──▶ UDP ──▶ IP ──▶ Network
[App3] ┘
```

- Demultiplexing ( 수신 측 )
  - 도착한 UDP 데이터그램을 Port 번호를 기준으로 해당 앱에 분배한다.
  - UDP 헤더의 목적지 port 확인

```
Network ──▶ IP ──▶ UDP ──▶
                    ├─ App1 (Port 5000)
                    ├─ App2 (Port 53)
                    └─ App3 (Port 443)
```

# 01 UDP ( User Datagram Protocol )

- Source and Destination Port



# 01 UDP ( User Datagram Protocol )

## 주요 장점

- 매우 간소화된 동작
- Header 크기가 작음 ( 8 Byte )
- 네트워크 대역폭이 작음
- 상태 유지 없음
- 낮은 지연성 ( Low Latency )

## 주요 단점

- ACK (Acknowledgement) 없음
  - 수신자가 "잘 받았다 " 는 응답을 보내지 않기 때문에 송신자는 도착 여부 확인 불가
- 전달 보장 없음
  - 패킷 유실 감지 불가
- 사전 연결 없음
  - 상대가 실제 존재하는지 확인 안함
- 흐름 제어 없음
  - 수신자의 처리 속도에 맞춰 전송 속도 조절 불가
- 혼잡 제어 없음
  - 네트워크가 붐빌 때 속도 조절 불가
- 순서 보장 없음
  - 패킷이 순서 없이 도작 가능

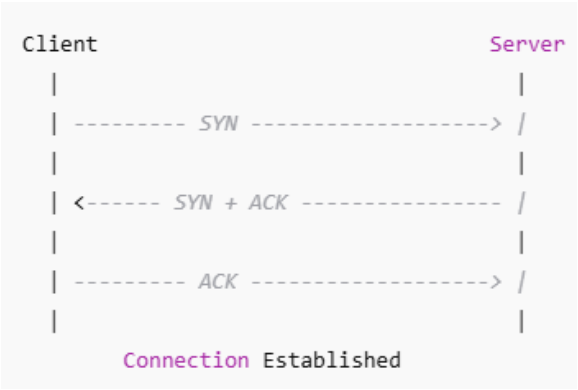
# 02 TCP ( Transmission Control Protocol )

## 개요

- **신뢰성 있는 데이터 전송을 보장하는 전송 계층 ( Layer 4) 프로토콜**
  - **"데이터를 반드시 정확하게, 순서 있게, 빠짐없이 보내고 싶을 때" 사용**
- UDT와 동일하게 Port 번호를 사용하여 프로세스를 선택
- Connection 기반 프로토콜
  - 반드시 사전 연결을 만든 후에 데이터 전송
- 상태 유지함
- TCP에서 주고 받는 데이터 단위를 Segment 라고 부름

## TCP Handshake ( 3-Way Handshake )

- 두 호스트가 데이터 전송을 시작하기 전에 "연결을 설정하는 절차" 이다.
- TCP 연결은 3단계 과정이 필요하다.
  - 1 단계: SYN ( 연결 요청 )
    - 클라이언트가 서버에게 연결 요청
  - 2 단계: SYN + ACK ( 연결 수락 )
    - 서버가 요청을 수락
  - 3 단계: ACK ( 연결 완료 )
    - 클라이언트가 서버 응답 확인



# 02 TCP ( Transmission Control Protocol )

## 주요 사용 사례

- 신뢰성 있는 통신 ( ex. 파일 전송, 금융 데이터 전송 등 )
- 원격 쉘 ( ex. SSH, Telnet 등 )
- **DB 연동**
- **웹 통신 ( HTTP, HTTPS )**
- 양방향 통신 ( ex. 채팅 서버 )

## Multiplexing & Demultiplexing

- Multiplexing ( 송신 측)
- Demultiplexing ( 수신 측 )

  UDP와 개념 동일.

# 02 TCP 장단점

## 장점

- **신뢰성 보장**
- **사전 합의 없이는 데이터 전송 불가**
- **흐름 및 혼잡 제어 가능**
- **순서 및 무결성 보장**

## 단점

- UDP에 비해서 Header 값이 크다.
- 대역폭이 증가
- 상태유지 기능으로 인한 메모리 소비
- 특정 작업에서는 높은 지연성 발생 가능

# 4. DNS

# 01 DNS ( Domain Name System )

## 개요

- **“사람이 사용하는 이름을 컴퓨터가 이해하는 IP 주소로 바꿔주는 시스템“**
  - **ex. www.google.com → 142.250.206.36**
- 분산 데이터베이스 시스템으로 전 세계에 흩어진 **서버들이 계층 구조로** 협력하며 서비스됨
- 하나의 도메인이 반드시 하나의 IP 가리키는 것은 아니고. 여러 IP를 가리킬 수 있음.
- DNS는 이름과 실제 서버(IP)를 분리하는 추상화 계층이기 때문에 서버 구조를 바꿔도 사용자는 영향이 없음.
- 위치에 기반한 가까운 서버 IP를 반환할 수 있기 때문에 지연 시간과 사용자 경험 개선 기대.

## Domain Name 구조

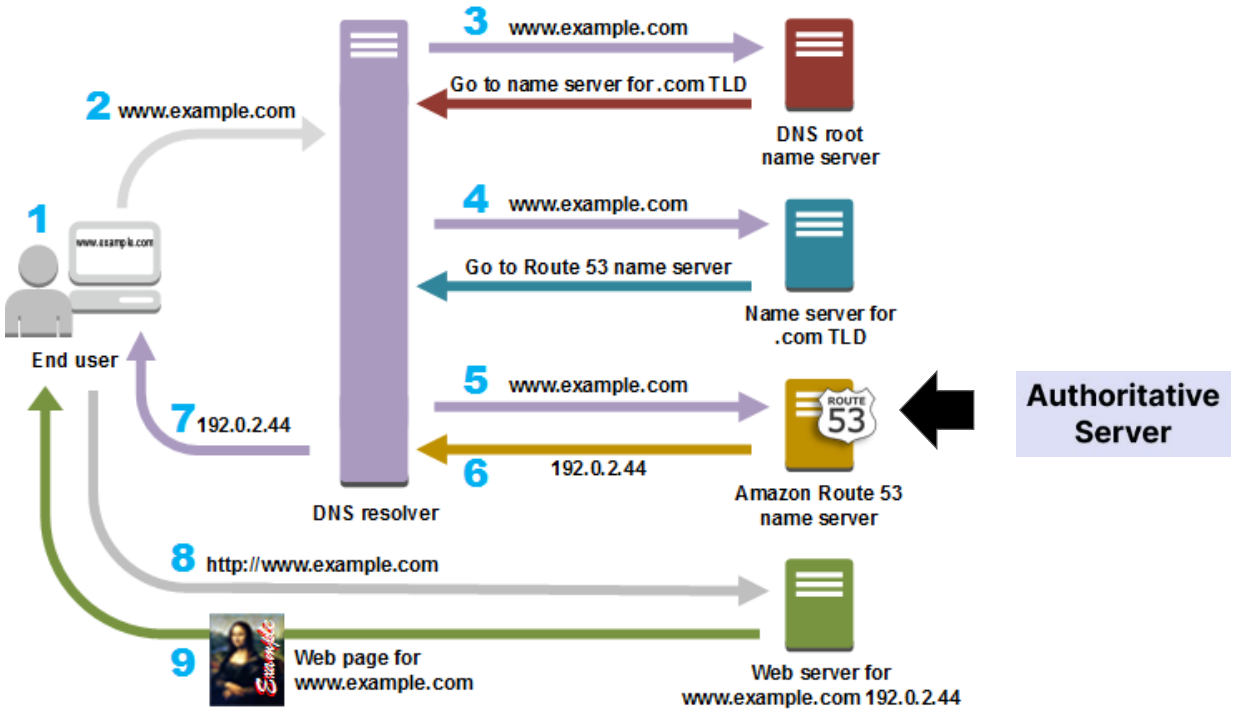
- 예> www.example.com.

| 구성요소 | 설명 |
|---|---|
| . | Root Domain |
| com | Top Level Domain (TLD ) |
| example | Second Level Domain |
| www | Sub Domain |

# 01 DNS ( Domain Name System )

## DNS 계층 구조 ( Hierarchy )

| 단계 | 설명 |
|---|---|
| 1 | Root DNS Server |
| 2 | TLD DNS Server ( .com, .net, .kr ) |
| 3 | Authoritative DNS Server |
| 4 | Local DNS Resolver ( ISP, 회사 ) |



# 01 DNS ( Domain Name System )

- DNS 지원 ( 가비아, AWS의 Route 53 등 )



# 5. 실습

# 01 실습1

## 목표

- 내 PC의 IP 주소를 확인하고, 서브넷 마스크를 이용해서 네트워크 주소와 사용 가능한 IP 범위를 알아보기
- CIDR: 192.168.0.0 /24

### 명령어: ipconfig /all

```
무선 LAN 어댑터 Wi-Fi:

   연결별 DNS 접미사. . . . :
   설명. . . . . . . . . . . . : Intel(R) Wi-Fi 6E AX211 160MHz
   물리적 주소 . . . . . . . . : 00-A5-54-E9-40-3F
   DHCP 사용 . . . . . . . . . : 예
   자동 구성 사용. . . . . . . : 예
   링크-로컬 IPv6 주소 . . . . : fe80::3731:91ab:a38d:6e8f%17(기본 설정)
   IPv4 주소 . . . . . . . . . : 192.168.0.4(기본 설정)
   서브넷 마스크 . . . . . . . : 255.255.255.0
   임대 시작 날짜. . . . . . . : 2026년 10월 23일 금요일 오전 8:20:09
   임대 만료 날짜. . . . . . . : 2026년 10월 23일 금요일 오후 7:20:42
   기본 게이트웨이 . . . . . . : 192.168.0.1
   DHCP 서버 . . . . . . . . . : 192.168.0.1
   DHCPv6 IAID . . . . . . . . : 338725194
   DHCPv6 클라이언트 DUID. . . : 00-01-00-01-2F-50-74-DA-00-A5-54-E9-40-3F
   DNS 서버. . . . . . . . . . : 210.220.163.82
                                 219.250.36.130
   Tcpip를 통한 NetBIOS. . . . : 사용
```

# 01 실습2

## 목표

- 내 컴퓨터에서 다른 장치까지 네트워크 통신이 가능한지, 응답이 얼마나 빠른지, 패킷이 손실되는지 등 확인.
- 상대방이나 네트워크 장비가 ping에 대해 응답하지 않도록 설정되어 있을 수 있음.
- ICMP(Internet Control Message Protocol) 이용.

### 명령어: ping -n 2 google.com

```
C:\Users\User>ping google.com

Ping google.com [172.217.209.101] 32바이트 데이터 사용:
172.217.209.101의 응답: 바이트=32 시간=37ms TTL=111
172.217.209.101의 응답: 바이트=32 시간=42ms TTL=111
172.217.209.101의 응답: 바이트=32 시간=39ms TTL=111
172.217.209.101의 응답: 바이트=32 시간=38ms TTL=111

172.217.209.101에 대한 Ping 통계:
    패킷: 보냄 = 4, 받음 = 4, 손실 = 0 (0% 손실),
왕복 시간(밀리초):
    최소 = 37ms, 최대 = 42ms, 평균 = 39ms

C:\Users\User>ping -n 2 google.com

Ping google.com [172.217.209.101] 32바이트 데이터 사용:
172.217.209.101의 응답: 바이트=32 시간=41ms TTL=111
172.217.209.101의 응답: 바이트=32 시간=43ms TTL=111

172.217.209.101에 대한 Ping 통계:
    패킷: 보냄 = 2, 받음 = 2, 손실 = 0 (0% 손실),
왕복 시간(밀리초):
    최소 = 41ms, 최대 = 43ms, 평균 = 42ms
```

# 01 실습3

- **목표**
  - DNS 서버에 도메인 주소 조회하기

**명령어: nslookup google.com**

```
C:\Users\User>nslookup google.com
서버:    bns1.hananet.net
Address:  210.220.163.82

권한 없는 응답:
이름:    google.com
Addresses:  2404:6800:400a:1007::65
          2404:6800:400a:1007::8b
          2404:6800:400a:1007::66
          2404:6800:400a:1007::8a
          142.251.118.100
          142.251.118.138
          142.251.118.139
          142.251.118.101
          142.251.118.113
          142.251.118.102
```

# 01 실습4

## 목표

- 내 컴퓨터에서 목적지까지 패킷이 이동하는 경로를 추적하는 명령어.

### 명령어: tracert google.com




```
C:\Users\User>tracert google.com

최대 30홉 이상의
google.com [172.217.209.139](으)로 가는 경로 추적:

  1     4 ms     1 ms     1 ms  192.168.0.1
  2     2 ms     6 ms     1 ms  192.168.55.1
  3     *        *        *     요청 시간이 만료되었습니다.
  4     *        *        *     요청 시간이 만료되었습니다.
  5     *        *        *     요청 시간이 만료되었습니다.
  6     *        *        *     요청 시간이 만료되었습니다.
  7     *        *        *     요청 시간이 만료되었습니다.
  8     *        *     ^C
```

# 5. DHCP

# 01 DHCP ( Dynamic Host Configuration Protocol )

## 개요

- 네트워크에 연결된 클라이언트에게 IP주소, 서브넷 마스크, 기본 게이트웨이, DNS 서버 등의 네트워크 설정 정보를 자동으로 할당해 주는 프로토콜.

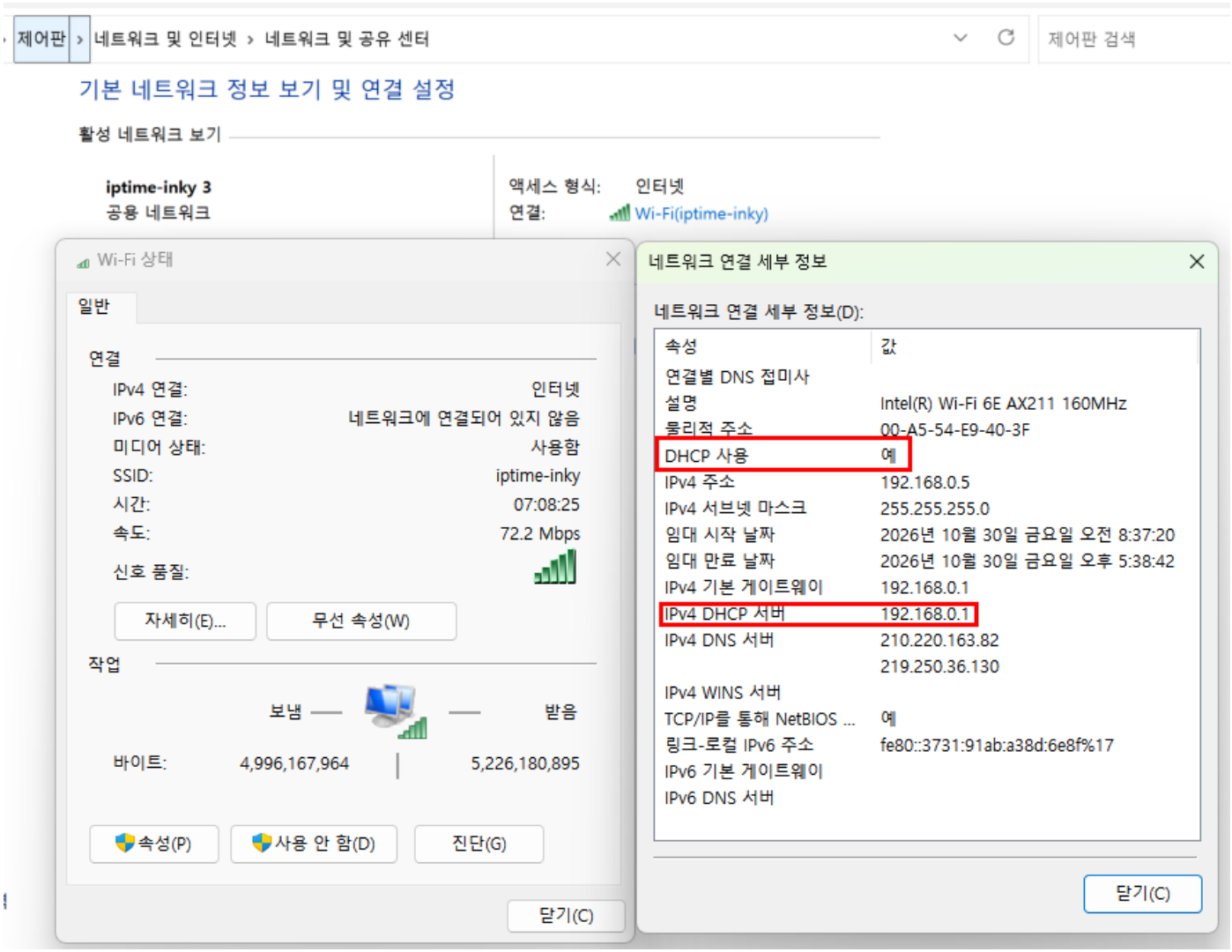
## 주요 목적

- IP 주소 자동 할당
- IP 주소 중복 방지
- 네트워크 설정 관리의 편리성
- IP 주소의 효율적인 재사용

## DHCP의 IP 주소 할당 과정 (DORA)



# 01 DHCP ( Dynamic Host Configuration Protocol )

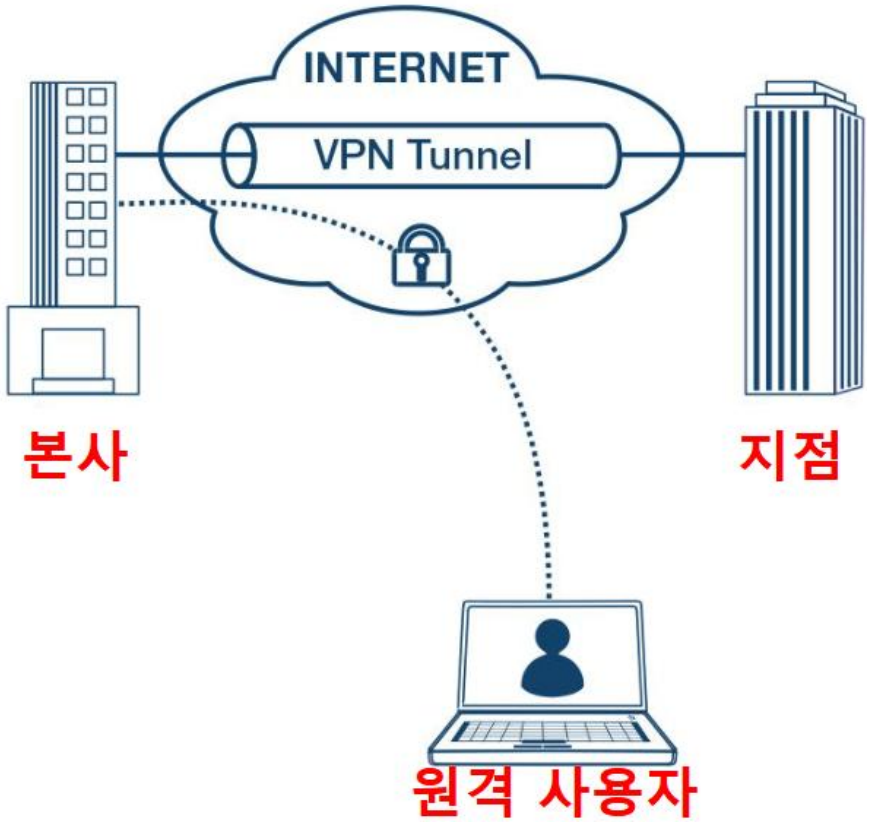


# 6. VPN

# 01 VPN ( Virtual Private Network )

## 개요

- 인터넷과 같은 공용 네트워크를 이용하면서도, 암호화된 연결 등을 통해서 떨어져 있는 컴퓨터나 네트워크를 사설 네트워크처럼 연결하는 기술임.



# 01 VPN ( Virtual Private Network )

## 활용

- 재택 근무 및 원격 접속
- 지사와 본사 연결
- 온프레미스와 클라우드 연결
- 공용 네트워크에서 통신 보호

## 장점

- 인터넷을 통해 원격 사용자가 내부 네트워크에 안전하게 접근하도록 지원.
- 공용 네트워크를 통과하는 트래픽 보호.
- 본사, 지사, 클라우드 간 연결 구성
- 전용 회선을 직접 구축하지 않고도 네트워크 간 연결 구현 가능.

# 6. 로드 밸런서

# 01 로드 밸런서 ( Load Balancer )

## 개요

- 여러 개의 서버를 구축하고, 사용자의 요청을 어느 서버로 보낼지 결정하는 역할.

### 1. 하나의 서버로 서비스를 운영하는 경우



# 01 로드 밸런서 ( Load Balancer )

## 2. 여러 대의 서버로 서비스를 운영하는 경우



# 01 로드 밸런서 ( Load Balancer )

## 주요 목적

- **부하 분산:** 요청이 특정 서버에 집중되는 것을 줄인다.
- **가용성 향상**: 일부 서버에 장애가 발생해도 정상 서버를 통해 서비스를 지속할 수 있게 한다.
- **확장성 향상**: 트래픽 증가에 맞춰 백엔드 서버를 추가하기 쉽게 한다.

## 부하 분산 알고리즘

| 알고리즘 | 동작 방식 | 적합한 상황 |
|---|---|---|
| Round Robin (기본) | 서버를 순서대로 선택 | 서버 성능이 비슷하고 요청이 균일한 경우 |
| Weighted Round Robin | 가중치에 따라 요청을 자동으로 분배 | 서버별 성능이 다른 경우 |
| Least Connections | 활성 연결 수가 적은 서버를 우선 선택 | 연결 유지 시간이 서로 다른 경우 |
| IP Hash | 클라이언트 IP를 기반으로 대상 선택 | 특정 클라이언트를 같은 서버로 유도해야 하는 경우 |

# 01 로드 밸런서 ( Load Balancer )

## OSI 계층과의 관계

**L7**
응용 계층

**Application Load Balancer**

HTTP 요청의 호스트 이름, URL 경로, 헤더 등 애플리케이션 정보를 바탕으로 요청을 분배할 수 있습니다.

예: `/api/*` 는 API 서버, `/admin/*` 은 관리자 서버

---

**L4**
전송 계층

**Network Load Balancer**

TCP 또는 UDP 같은 전송 프로토콜과 연결 정보를 중심으로 트래픽을 분산합니다.

예: TCP 연결을 여러 백엔드 서버로 분산

# 7. 암호화

# 1. 대칭키/비대칭키

## 개요

- 대칭키와 비대칭키는 데이터를 암호화하고 복호화할 때 사용하는 암호화 방식임.
- 네트워크, 웹 보안, HTTPS, SSL/TLS, SSH, 전자서명 등을 이해하기 위해서 반드시 알아야 되는 개념

## 용어

- 평문 ( Plaintext ) : 암호화하기 전의 원본 데이터
- 암호문 (Ciphertext) : 암호화된 데이터
- 암호화 (Encryption) : 평문을 암호문으로 변환하는 과정
- 복호화 (Decryption) : 암호문을 평문으로 되돌리는 과정
- 키(Key) : 암호화 및 복호화 과정에서 사용하는 비밀 정보

# 1. 대칭키/비대칭키

## ■ 암호화와 복호화의 기본 구조



# 2. 대칭키 암호화 ( Symmetric Key Encryption )

## 대칭키

- 암호화할 때 사용하는 키와 복호화를 사용하는 키가 동일한 암호화 방식.
- 송신자와 수신자가 동일한 비밀키(Secret Key)를 공유.



# 2. 대칭키 암호화 ( Symmetric Key Encryption )

## 장점

- 암호화와 복호화 속도가 빠르다.
- 대용량 데이터 암호화에 적합하다.
- 컴퓨터 자원 사용량이 상대적으로 적다.

## 단점

- 암호화 키를 상대방에게 안전하게 전달해야 한다.
- 키가 유출되면 해당 키로 암호화된 데이터가 위험해진다.
- 통신 상대가 많아지면 키 관리가 복잡해진다.

# 2. 대칭키 암호화 ( Symmetric Key Encryption )

## ■ 대표적인 대칭키 알고리즘

- 일반적으로 AES(Advanced Encryption Standard) 사용.

| 알고리즘 | 설명 |
|---|---|
| AES | 현재 가장 널리 사용되는 표준 대칭키 암호.<br>ex. AES-128 : 128 비트 키 사용<br>AES-192 : 192 비트 키 사용<br>AES-256 : 256 비트 키 사용 |
| DES | 오래된 암호화 알고리즘, 현재는 안전하지 않음. |
| 3DES | DES를 세 번 적용한 방식. 현재는 사용 중단 권고. |
| ChaCha20 | 빠르고 효율적인 스트림 암호. |

# 3. 비대칭키 암호화 ( Asymmetric Key Encryption )

## ■ 대칭키

- 암호화할 때 사용하는 키와 복호화를 사용하는 키가 서로 다른 암호화 방식.
- 두 개의 키를 한 쌍으로 사용.
  - 공개키 (Public Key) : 다른 사람에게 공개해도 되는 키
  - 개인키 (Private Key) : 소유자만 안전하게 보관해야 하는 키



# 3. 비대칭키 암호화 ( Asymmetric Key Encryption )

## 장점

- 개인키를 상대방에게 전달할 필요가 없다.
- 공개키를 공개해도 개인키가 안전하면 복호화할 수 없다.
- 전자서명과 인증에 활용할 수 있다.

## 단점

- 대칭키보다 암호화 및 복호화 연산이 느리다.
- 상대적으로 연산 자원을 많이 사용한다.
- 공개키가 정말 해당 사용자 또는 서버의 것인지 확인해야 한다.

# 3. 비대칭키 암호화 ( Asymmetric Key Encryption )

## ■ 대표적인 비대칭키 알고리즘

- 일반적으로 RSA 사용.

| 알고리즘 | 설명 |
|---|---|
| RSA | 공개키와 개인키를 사용하는 대표적인 암호 알고리즘 |
| ECC | 타원곡선 암호 방식 |
| ECDH | 타원곡선을 이용한 키 교환 방식 |
| ECDSA | 타원곡선을 이용한 전자서명 방식 |

# 4. 전자 서명 (Digital Signature )

## 전자서명

- 비대칭키의 또 다른 기능.



# 5. 대칭키와 비대칭키

## ■ 대칭키와 비대칭키 비교

| 비교 항목 | 대칭키 | 비대칭키 |
|---|---|---|
| 키 개수 | 하나의 비밀키 | 공개키와 개인키 한 쌍 |
| 암호화 키 | 비밀키 | 수신자의 공개키 |
| 복호화 키 | 동일한 비밀키 | 수신자의 개인키 |
| 속도 | 빠름 | 상대적으로 느림 |
| 키 전달 | 안전한 공유 필요 | 공개키는 공개 가능 |
| 주요 용도 | 데이터 암호화 | 키 교환, 암호화, 전자서명 |
| 대표 알고리즘 | AES | RSA, ECC 계열 |

# 5. 대칭키와 비대칭키

## ■ 대칭키와 비대칭키 실제 사례

| 기술 | 사용 방식 | 활용 목적 |
|---|---|---|
| HTTPS | 비대칭키 기술 + 대칭키 암호화 | 웹 통신 보안 |
| SSH | 공개키 기반 인증 + 키교환 + 대칭키 암호화 | 원격 서버 접속 및 통신 보호 |
| VPN | 키 교환 + 대칭키 암호화 | 네트워크 통신 보호 |
| 전자서명 | 비대칭키 기반 서명 + 해시 함수 | 사용자 및 데이터 무결성 검증 |
| 파일암호화 | 주로 대칭키 암호화 | 파일 보호 |
| TLS 프로토콜 | 인증 및 키 교환 + 대칭키 암호화 | 안전한 통신 채널 제공 |

# 6. HTTPS 통신

## HTTP vs HTTPS

| 구분 | HTTP | **HTTPS** |
|---|---|---|
| 전체 명칭 | HyperText Transfer Protocol | **HyperText Transfer Protocol Secure** |
| 기본 포트 | 80 | **443** |
| 데이터 암호화 | 기본적으로 암호화하지 않음 | **TLS로 암호화** |
| 서버 인증 | 기본 제공하지 않음 | **인증서 기반 서버 인증** |
| 데이터 무결성 | 기본 제공하지 않음 | **TLS를 통해 보장** |
| 보안 수준 | 낮음 | **높음** |

# 6. HTTPS 통신

- **https://example.com 요청 사례**



# 수고하셨습니다